PROBLEM SET / SEVEN

Luogu Problem Analysis — Seven Problems

洛谷七题解析

二维数组 · 数组遍历 · 排序统计 · 背包 · 贪心

每道题用三页讲透：这一页看清题面、下一页拆开思路、最后一页逐行读代码。

七题覆盖五个专题，题号均已与既往课件比对，无重复。

题目来源

洛谷 luogu.com.cn | 7 题 · 难度 入门 — 普及- | 每题 3 页

主讲 · 信息教研组 | 授课班级 · 中预 2026 级 B 班

七题地图

信息学竞赛培训 · 洛谷七题解析

题目与考点一览

七道题按「二维数组 → 数组遍历 → 排序统计 → 背包 → 贪心 → 二维模拟」的顺序排列，前后互为铺垫。每题的解析都落在第 3 到第 5 页之间，可按题号跳转。

1	P5730	显示屏	普及−	二维数组	打表 + 行优先输出	第 03 – 05 页
2	P1428	小鱼比可爱	入门	数组遍历	双重循环 + 条件累加	第 06 – 08 页
3	P1097	统计数字	普及−	排序统计	排序后块状计数	第 09 – 11 页
4	P1616	疯狂的采药	普及−	背包 · 完全	内层容量正序枚举	第 12 – 14 页
5	P1049	装箱问题	普及−	背包 · 01 变式	内层容量倒序枚举	第 15 – 17 页
6	P5723	质数口袋	入门	贪心	取最小质数 + 筛法	第 18 – 20 页
7	P1789	插火把	普及−	二维数组 + 模拟	矩阵标记 + 计数	第 21 – 23 页

例题一 · 读懂题面

P5730

显示屏

普及−

液晶屏上每个阿拉伯数字都能显示成 3 列 × 5 行的点阵，其中 X 表示亮点、. 表示暗点。给一串数字，把它整体打印出来，数字之间要留一列间隔。

输入格式

第一行一个正整数 n，表示数字的位数。

第二行一个长度为 n 的自然数串，数字之间不含空格。

输出格式

共 5 行，每行长度是 3 × n + (n − 1)。

即第 1 个数字到第 2 个数字之间插一列点，依此类推。

数据范围

1 ⩽ n ⩽ 100。

数字串中只含 0 − 9，可能以 0 开头。

样例

```
10
0123456789
```

```
XXX...X.XXX.XXX.X.X.XXX.XXX.XXX.XXX.XXX
X.X...X...X...X.X.X.X...X.....X.X.X.X.X
X.X...X.XXX.XXX.XXX.XXX.XXX...X.XXX.XXX
X.X...X.X.....X...X...X.X.X...X.X.X...X
XXX...X.XXX.XXX...X.XXX.XXX...X.XXX.XXX
```

样例把 0 到 9 依次排开，正好是一张完整的字形表。

注意第 4 行第 1 列起是「X.X」，而数字 1 的那一列是「..X」——数字 1 是右对齐，不是居中，这一点很容易画错。

题意建模的一句话

这不是一道「算」的题，而是一道「查表 + 拼接」的题：先记住 10 个数字各自的字形，再按题目的拼接规则把字块拼成 5 行长串。

例题一 · 思路拆解

字形是题目给定的常量，不可能在运行时推导出来。所以整道题只有一个关键动作：把字形存好，然后按行拼接。

第一步：认清数据的维度

每个数字的字形是 5 行 × 3 列，共 10 个数字。因此最自然的存储是一张三维表 g[10][5][3]，或等价的字符串表 string g[10][5]。

第二步：把字形表打出来

逐个数把 5 行字符合手敲进常量数组。注意数字 1 的亮点全在最右列（..X），数字 4 的第 1 行是 X.X —— 这些细节必须与样例逐字核对。

第三步：确定输出的循环顺序

题目要的是 5 行整体画面，所以最外层枚举「行 r」（0 到 4），内层枚举「第几个数字 j」（0 到 n−1），再输出该数字第 r 行的 3 个字符。

第四步：处理数字间的间隔

每输出完一个数字，如果它不是最后一个，就补一个 .。写成「先输出数字，再在 j 不足 n−1 时补点」，可保证行尾不多点。

最容易搞反的地方：循环的层级

很多同学第一反应是「外层遍历数字、内层遍历 5 行」，这样会先把第一个数字整块打完再打第二个 —— 但题目要的是 5 行整体画面，必须「外层行、内层数字」。这是本题唯一的思维门槛。

例题一 · 参考代码


```cpp
#include <iostream>
#include <string>
using namespace std;

// g[d][r]：数字 d 在第 r 行的 3 个字符
string g[10][5] = {
  {"XXX","X.X","X.X","X.X","XXX"},  // 0
  {"..X","..X","..X","..X","..X"},  // 1 右对齐
  {"XXX","..X","XXX","X..","XXX"},  // 2
  {"XXX","..X","XXX","..X","XXX"},  // 3
  {"X.X","X.X","XXX","..X","..X"},  // 4
  {"XXX","X..","XXX","..X","XXX"},  // 5
  {"XXX","X..","XXX","X.X","XXX"},  // 6
  {"XXX","..X","..X","..X","..X"},  // 7
  {"XXX","X.X","XXX","X.X","XXX"},  // 8
  {"XXX","X.X","XXX","..X","XXX"}   // 9
};

int main() {
    int n;
    string s;
    cin >> n >> s;
    for (int r = 0; r < 5; r++) {          // 外层：5 行
        for (int j = 0; j < n; j++) {      // 内层：n 个数字
            int d = s[j] - '0';
            cout << g[d][r];               // 数字 d 的第 r 行
            if (j != n - 1) cout << ".";   // 非末位补间隔列
        }
        cout << "
";
    }
    return 0;
}
```

字形表怎么声明

string g[10][5] 最直观：g[d][r] 就是「数字 d 的第 r 行」。也可写成三维 char 数组，但字符串写法更省心。

两个循环的层级

外层 r 控行、内层 j 控数字，这是本题的骨架。写反了画面就变成「一个数字一块」而不是 5 行长条。

间隔列的位置

先输出数字、再判断补点，能保证行尾不多输出一个 .。判断条件是 j 不等于 n−1，而不是 n。

字符映射的小技巧

s[j] − '0' 把字符 '0' 到 '9' 映射成整数 0 到 9。这是 C++ 里字符转数字的标准写法，比 switch 分支简洁得多。

例题二 · 读懂题面

P1428 小鱼比可爱

入门

n 只鱼从左到右排成一排，头都朝左，所以每只鱼只能看见它左边的鱼。给出每只鱼的可爱程度，求每只鱼左边有多少只鱼比它可爱程度小。

输入格式

第一行一个正整数 n，表示鱼的数目。

第二行 n 个非负整数，用空格隔开，依次表示从左到右每只鱼的可爱值 ai。

输出格式

一行 n 个整数，用空格隔开。

第 i 个数表示第 i 只鱼左边比它可爱值小的鱼的个数。

数据范围

$1 \leqslant n \leqslant 100$

$0 \leqslant ai \leqslant 10$

注意可爱值可能相等，相等不算「不如」。

样例

```
6
4 3 0 5 1 2
```

```
0 0 0 3 1 2
```

第 1 至第 3 只鱼左边都没人比它小，所以是 0。

第 4 只鱼可爱值 5，左边 4、3、0 都比它小，得 3。

第 5 只鱼可爱值 1，左边只有 0 比它小，得 1。

第 6 只鱼可爱值 2，左边 0 和 1 比它小，得 2。

题意建模的一句话

对每一只鱼，回头清点它左边的所有数字，数一数其中「严格小于」自己的有多少个。这是一个「逐位置向左扫描」的统计问题。

例题二 · 思路拆解

数据规模只有 n ⩽ 100，最朴素的平方级做法完全够用，关键是把「向左看」的方向写对。

第一步：确认扫描方向

鱼头朝左，所以第 i 只鱼只能看见下标比它小的鱼。内层循环的下标 j 必须严格小于 i，写成大于 i 就会统计到右边的鱼。

第二步：确定比较条件

题目问的是「不如自己可爱」，即严格小于。判断条件写成 a[j] 小于 a[i]，用小于等于就会把相等的情况多算进去。

第三步：计数器放在哪

计数变量必须在外层循环内部、内层循环之前声明并初始化为 0。写在循环外面会跨鱼累加，导致后面的结果全部偏大。

第四步：输出与格式

得到结果后可以直接 printf 或 cout 输出并跟一个空格。最后一个数后面的空格在本题不影响判题，但规范写法是判断后不输出多余空格。

一个值得知道的进阶角度

可爱值只有 0 到 10 这 11 种可能，可以用一个长度为 11 的桶数组边扫边累计：「左边比自己小的个数」= 桶里下标 0 到 a[i] − 1 的和。这样一趟 O(n) 就能算完，是本题的优化版。

例题二 · 参考代码


```cpp
#include <cstdio>

int a[105];                     // 每只鱼的可爱值

int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n; i++) {         // 逐只鱼
        int cnt = 0;                      // 每只鱼单独清零
        for (int j = 0; j < i; j++) {     // 只扫左边
            if (a[j] < a[i])              // 严格小于
                cnt++;
        }
        printf("%d", cnt);
        if (i != n - 1) printf(" ");      // 行尾不留空格
    }
    printf("
");
    return 0;
}
```

内层上界是 i 不是 n

写成 j 小于 n 就会把右边的鱼也算进来。本题鱼头朝左，只允许 j 严格小于 i，这是第一位的正确性条件。

cnt 必须写在循环里

如果写成全局变量或放在外层循环之外，各只鱼的计数会互相累加，前面的结果会「污染」后面的结果。

比较用严格小于

题目中可爱值可以相等，相等不算「不如」。用 a[j] 小于 a[i]；写成小于等于会把相等的鱼也数进去。

复杂度完全够用

n 最大 100，外层 100 次、内层平均 50 次，总共约 5000 次比较，任何写法都能轻松通过。这一题的正确姿势是先把逻辑写对，不必过早考虑优化。

例题三 · 读懂题面

P1097

统计数字

普及−

给 n 个自然数，统计每个不同的数各自出现了几次，并按照数值从小到大的顺序输出。数值可能很大、重复可能很多。

输入格式

第一行一个整数 n。

接下来 n 行，每行一个自然数。注意是每行一个，不是同一行用空格隔开。

输出格式

共 m 行（m 为不相同的数的个数）。

每行两个整数：数值 和 出现次数，中间用一个空格隔开。

数据范围

$n \leqslant 200000$，每个数不超过 1.5×10^9 。

不相同的数不超过 10^4 个 —— 这是解题的关键提示。

样例

```
8
2
4
2
4
5
```

```
2 3
4 2
5 1
100 2
```

原数组中 2 出现 3 次、4 出现 2 次、5 出现 1 次、100 出现 2 次。

输出时按数值升序排列，所以先输出 2，再 4、5，最后 100。

题意建模的一句话

「统计每个人出现几次 + 按数值排序输出」这两个需求合在一起，最自然的解法就是：先排序，让相同的数聚在一起，然后一趟扫描把它们成块地数出来。

例题三 · 思路拆解

为什么不用开一个 1.5×10^9 大的计数数组？因为数值太大开不下。为什么不一定要用 map？因为排序法的常数更小、更好写。

第一步：选一个够大的数据类型

数值上限 1.5×10^9 已经接近 int 的边界（约 2.147×10^9 ）。为了保险，数组统一用 long long，读入也用 long long。

第二步：读入后排序

读入 n 个数后调用 sort(a, a + n) 升序排序。排序之后，所有相同的数必然紧挨在一起，这是后续计数的全部依据。

第三步：一趟扫描做块状计数

设一个计数器 cnt 从 1 开始。从左往右扫，若当前数与后一个数相同就 cnt++；若不同，说明这一块结束了，输出「当前数 + cnt」，然后把 cnt 重置为 1。

第四步：别漏掉最后一组

循环走到倒数第二个数就结束了，最后一组的统计没有机会输出。解决办法有两种：循环后单独补输出一次；或者把 a[n] 预置成一个哨兵值（例如 −1）再去扫。

本题真正想训练的能力

不是「会不会用 map」，而是「能不能看出排序之后序列的结构」——相同的元素必然相邻。看出这一点，统计就退化成一趟线性扫描，比哈希表更省内存、常数更小。

例题三 · 参考代码


```cpp
#include <cstdio>
#include <algorithm>
using namespace std;

long long a[200005];            // 数值接近 int 上限，用 long long

int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%lld", &a[i]);

    sort(a, a + n);             // 相同元素必然相邻

    int cnt = 1;                // 当前这一组已数到的个数
    for (int i = 1; i <= n; i++) {
        if (i < n && a[i] == a[i - 1]) {
            cnt++;              // 还在同一组里
        } else {
            printf("%lld %d
", a[i - 1], cnt);   // 输出整组
            cnt = 1;            // 重置，开始下一组
        }
    }
    return 0;
}
```

哨兵写法的妙处

循环上界写成 i ⩽ n，并在判断中加入 i 小于 n 的守卫。当 i 等于 n 时守卫为假，必然落入 else 分支，最后一组就顺理成章地输出了。

为什么要用 long long

数值可达 1.5×10^9 ，int 的上限约 2.147×10^9 ，虽然勉强装得下，但一旦题目数据加强就会溢出。统一用 long long 是零成本的保险。

输出格式对应关系

printf 里输出的是 a[i − 1] 而不是 a[i]。因为 a[i] 已经是新一组的值了，当前这一组的代表值是它的前一个。这不是笔误，是必须的。

两种正确的收尾方式

方式一（本页代码）：循环上界取 n，用 i 小于 n 做守卫，让最后一组自然落入 else。方式二：循环只到 n − 1，循环结束后再补一句输出 a[n − 1] 和 cnt。任选其一，但不要两个都不做。

例题四 · 读懂题面

P1616 疯狂的采药

普及−

与经典的「采药」几乎一样，唯一的区别是：每一种草药都可以采摘无限多次。给定总时间与每种草药的耗时、价值，求时间限制内能获得的最大总价值。

输入格式

第一行两个整数 t 与 m，分别表示总时间与草药种类数。

接下来 m 行，每行两个整数 ai 与 bi，表示第 i 种草药的采摘耗时与价值。

输出格式

一行一个整数，表示在规定时间内可以采到的最大总价值。

答案可能超过 int 范围，建议使用 long long。

数据范围

$1 \leqslant m \leqslant 10^4$ ，$1 \leqslant t \leqslant 10^7$

且 $m \times t \leqslant 10^7$

$1 \leqslant ai, bi \leqslant 10^4$

样例

```
70 3
71 100
69 1
1 2
```

```
140
```

第 1 种草药要 71 个时间，超过总时间 70，一次都采不了。

第 3 种草药耗时 1、价值 2，性价比最高：70 个时间全部拿它，

可以采 70 次，总价值 2 × 70 = 140。这就是「无限次」带来的差别。

题意建模的一句话

「有时间上限 + 每种草药可取无限多次 + 求最大收益」—— 三个条件同时满足，就是完全背包。与 01 背包的唯一差别是：每件物品可以重复选取任意多次。

例题四 · 思路拆解

完全背包的方程长得和 01 背包一模一样，唯一的区别藏在内层循环的方向里。这一页要把「为什么正序就对了」讲清楚。

第一步：识别出完全背包

「每种草药可以无限制地疯狂采摘」这句话就是信号。只要物品可以重复取，就是完全背包，不是 01 背包。

第二步：写出状态与转移

dp[j] 表示总时间为 j 时能取得的最大价值。转移式 dp[j] = max(dp[j], dp[j − a[i]] + b[i]) —— 形式上与 01 背包完全一致。

第三步：把内层循环改成正序

这是全题唯一需要动手改的地方：j 从 a[i] 递增到 t。正序时 dp[j − a[i]] 读到的是本轮已经更新过的值，等价于「这件物品已经在包里放过了，还能再放」。

第四步：开够空间、用 long long

t 最大 10^7 ，dp 数组要开到 10^7 + 1 级别（约 40 MB，需声明为全局数组）。价值可能很大，dp 建议用 long long 或确认 int 足够。

一句话记住方向

01 背包倒序、完全背包正序。倒序保证每件物品最多拿一次，正序允许同一件物品反复拿 —— 循环方向决定背包类型，这是背包问题的第一个分水岭。

例题四 · 参考代码


```cpp
#include <cstdio>
#include <algorithm>
using namespace std;

const int MAXT = 10000005;
int a[10005], b[10005];          // 耗时、价值
long long dp[MAXT];              // 全局数组，自动清零且不占栈

int main() {
    int t, m;
    scanf("%d %d", &t, &m);
    for (int i = 1; i <= m; i++)
        scanf("%d %d", &a[i], &b[i]);

    for (int i = 1; i <= m; i++) {
        for (int j = a[i]; j <= t; j++) {     // 正序！完全背包
            long long cand = dp[j - a[i]] + b[i];
            if (cand > dp[j]) dp[j] = cand;
        }
    }
    printf("%lld
", dp[t]);
    return 0;
}
```

内层正序是关键

j 从 a[i] 一路增到 t。循环下界直接取 a[i]，比 a[i] 更小的容量连一件都放不下，不必计算。

数组必须是全局的

t 可达 10^7 ，dp 约 40 – 80 MB。写在 main 里面会占用栈空间，容易直接崩溃；写成全局变量则分配在静态区，安全得多。

累加写成 if 而不是 max

内层执行约 10^7 次，把 max 函数调用展开成一次 if 比较，能省下可观的常数。数据规模大时这个细节有意义。

关于 long long

每种草药价值上限 10^4 ，时间上限 10^7 ，理论上最大总价值可达 10^{11} 量级，已超过 int。稳妥做法是 dp 与输出都用 long long，读入 printf 时用 %lld。

例题五 · 读懂题面

P1049 装箱问题

普及−

有一个容量为 V 的箱子，n 件物品各有体积。从中任取若干件装入箱内（也可以一件都不取），使箱子的剩余空间最小。求这个最小剩余空间。

输入格式

第一行一个整数 V，表示箱子容量。

第二行一个整数 n，表示物品总数。

接下来 n 行，每行一个正整数，表示该物品的体积。

输出格式

一行一个整数，表示箱子最小的剩余空间。

注意输出的是「剩余」，不是「装进去的总体积」。

数据范围

0 小于 n，n ⩽ 30

1 ⩽ V ⩽ 20000

每件物品体积为正整数

样例

```
24
6
8
3
12
7
```

```
0
```

容量 24。取 12 + 3 + 9 = 24，恰好装满，剩余 0。

另一组取法 12 + 3 + 9 与 8 + 7 + 9 都能凑满，答案都是 0。

此时 dp[24] = 24，输出 24 − 24 = 0。

题意建模的一句话

「剩余空间最小」等价于「装进去的总体积最大」。而本题的每件物品体积恰好等于它的价值 —— 于是它就是一道标准 01 背包，只是最后要再做一次减法。

例题五 · 思路拆解

这道题的价值在于「把求最小值的问题翻译成求最大值」。翻译完成了，剩下的就是默写 01 背包模板。

第一步：把目标翻译过来

设装进去的总体积为 S。剩余空间就是 V − S。要让剩余最小，就是让 S 最大。于是问题变成「容量 V 的背包，最多能装多少体积」。

第二步：让价值等于体积

只要令 v[i] = w[i] = 物品体积，普通 01 背包的「最大价值」就自动变成了「最大装入体积」。这是本题建模的核心一步。

第三步：套用 01 背包模板

dp[j] = max(dp[j], dp[j − v[i]] + v[i])，内层 j 从 V 倒序枚举到 v[i]。因为每件物品最多装一次，所以必须倒序。

第四步：答案是 V 减去 dp[V]

dp[V] 是容量 V 下能装入的最大体积，最终输出 V − dp[V]。忘了这步减法、直接输出 dp[V]，是本题最高频的失分点。

与例题四对照着记

例题四是完全背包（内层正序），本题是 01 背包（内层倒序）。两道题放在一起，正好是「循环方向决定背包类型」的最佳对照练习 —— 背下这两行循环，比背十个公式都有用。

例题五 · 参考代码


```cpp
#include <cstdio>
#include <algorithm>
using namespace std;

int v[35];                      // 每件物品的体积
int dp[20010];                  // dp[j]：容量 j 下能装入的最大体积

int main() {
    int V, n;
    scanf("%d %d", &V, &n);
    for (int i = 1; i <= n; i++)
        scanf("%d", &v[i]);

    for (int i = 1; i <= n; i++) {
        for (int j = V; j >= v[i]; j--) {    // 倒序！01 背包
            dp[j] = max(dp[j], dp[j - v[i]] + v[i]);
        }
    }
    printf("%d
", V - dp[V]);          // 输出剩余空间
    return 0;
}
```

价值就是体积

dp[j - v[i]] + v[i] 里的加项就是 v[i]，因为本题的价值与体积相等。不用额外开一个价值数组，代码短了一半。

内层必须倒序

写成正序会让同一件物品被重复装入，答案会变成 0（箱子被无限塞满）。样例的输入 5 与单件体积 1 是经典的验证用例。

最后一步减法

输出 V - dp[V]，不是 dp[V]。判题时如果得到一个和期望值相去甚远的正数，先检查这一行。

一个容易忽略的边界

如果每件物品的体积都大于 V，那么一件也装不进去，dp[V] 保持为 0，输出 V - 0 = V，即箱子原封不动。这正是题目允许的「也可以不取」的情况。

例题六 · 读懂题面

P5723 质数口袋

入门

有一个质数口袋。从 2 开始，依次判断每个自然数是不是质数，是质数就装进口袋。口袋里所有质数之和不能超过 L。求最多能装下几个质数，并把它们从小到大输出。

输入格式

一行一个正整数 L，表示口袋的承重量。

即口袋里质数之和的上限。

输出格式

先依次输出装进口袋的质数，每个占一行。

最后一行输出装下的质数个数。

数据范围

$1 \leqslant L \leqslant 10^5$

质数从小到大取，每个质数只取一次。

样例

```
100
```

```
2
3
5
7
11
13
```

从 2 起依次取质数并累加：

2+3+5+7+11+13+17+19+23 = 100，刚好装满。

再加下一个质数 29 就超过 100 了，所以停在这里。

一共 9 个质数，最后输出 9。

题意建模的一句话

要装的数量尽可能多，就应该每次都挑当前能挑的最小的质数 —— 这正是贪心策略，而且在这道题里局部最优恰好就是全局最优。

例题六 · 思路拆解

为什么贪心在这里是对的？因为所有质数都是正数，用小质数换掉一个大质数，同样的和能装下更多个数。所以从小到大取一定最优。

第一步：说明贪心的正确性

假设最优方案里有一个较大的质数，而它前面还有一个没用上的较小质数。把大的换成长的一定不会让总和变大，因此可以把方案调整成「从小到大取」。

第二步：预筛出 L 以内的质数

用埃氏筛：先把 2 到 L 全部标记为质数，然后对每个质数 p，把 p 的所有倍数（从 2p 起）标记为非质数。时间复杂度约 $O(L \log \log L)$，$L = 10^5$ 时非常快。

第三步：从小到大取，边取边累加

遍历 2 到 L，遇到质数就累加。注意题目要求「装满不超过 L」，所以约束是小于等于，取到恰好等于时仍然算装进去了。

第四步：判超限要放在累加之前

必须先判断「当前和 + 这个质数」是否超过 L，确认不超再累加、再计数、再输出。放在累加之后判断，就会多装一个质数。

本题最容易错的地方

「先累加再检查」和「先检查再累加」看起来只差一行，结果却差一个质数。因为质数一旦装进口袋就不能再拿出来，检查必须发生在「决定装它」之前。

例题六 · 参考代码

P5723 · 贪心 + 埃氏筛

```cpp
#include <cstdio>

const int MAXL = 100005;
bool isp[MAXL];                 // 标记是否为质数

int main() {
    int L;
    scanf("%d", &L);

    for (int i = 2; i <= L; i++) isp[i] = true;    // 先全当质数
    for (int i = 2; i <= L; i++)
        if (isp[i])
            for (int j = 2 * i; j <= L; j += i)
                isp[j] = false;                    // 筛掉合数

    int sum = 0, cnt = 0;
    for (int i = 2; i <= L; i++) {
        if (isp[i] == false) continue;
        if (L - sum < i) break;       // ★ 先判断，再累加
        sum += i;
        cnt++;
        printf("%d
", i);
    }
    printf("%d
", cnt);
    return 0;
}
```

筛法从 i × i 开始

因为比 i × i 小的 i 的倍数，一定已经被更小的质因子筛过了。从 i × i 起筛是标准优化，能省掉近一半无效标记。

注意循环上界

外层 i 只筛到根号 L 就够了（写成 i 不超过 L 除以 i 可避免乘法溢出）。超过根号 L 的合数必然有一个不超过根号 L 的因子，早已被筛掉。

判断与累加的顺序

在循环里必须「先判断、再累加」。写法上把「剩余可装容量」作为比较对象：若剩余容量已经不够装下这个质数，就直接跳出循环。这个顺序一旦颠倒，就会多装一个质数。

关于 break 与 continue

continue 用于跳过合数（不中断当前循环），break 用于遇到装不下的质数时终止整个循环。两者含义不同，不能互换 —— 写反了输出会变得毫无规律。

例题七 · 读懂题面

P1789 插火把

普及−

在一个 n × n 的方阵里放 m 个火把与 k 个萤石。没有光照、也没有放置物品的格子会生成怪物。求有多少个格子会生成怪物。

输入格式

第一行三个整数 n、m、k，分别表示方阵边长、火把数、萤石数。

接下来 m 行是火把坐标。

再接下来 k 行是萤石坐标。

输出格式

一行一个整数，表示会生成怪物的格子数量。

光照范围超出方阵的部分直接忽略。

数据范围

$1 \leqslant n \leqslant 100$

$1 \leqslant m + k \leqslant 25$，其中 $m \leqslant 25$、$0 \leqslant k \leqslant 5$

一定有火把，可能没有萤石。

样例

```
5 1 0
3 3
```

```
12
```

5 × 5 = 25 个格子。中心 (3,3) 放一个火把，能照亮 13 格。

25 − 13 = 12，即有 12 个格子没有光照会生成怪物。

注意火把所在的格子本身也被照亮，不算怪物。

题意建模的一句话

这是一道纯粹的模拟题：开一张 n × n 的布尔表表示「是否被照亮」，每放一个光源就按它的形状把对应格子置为真，最后数一数还有多少格是假的。

例题七 · 思路拆解

本题没有任何算法，就是老老实实按题目给的形状一格一格地标记。难点全在「火把的照明范围到底长什么样」。

第一步：开表并想清初值

开一个 n + 5 见方的布尔数组 light，全部初始化为 false（未照亮）。多开几格是为了后面标记时的越界防护留出余量。

第二步：逐个处理火把

火把的照明范围是 13 个格子：以自身为中心的上下左右各延伸 2 格（共 9 格，形成一个十字），再加上四个对角各 1 格。这 13 个相对偏移必须与题目的图示逐格核对。

第三步：逐个处理萤石

萤石是以自身为中心的 5 × 5 实心方块，共 25 格。用一个双重循环 i、j 从 −2 到 2 遍历偏移量即可，比手写 25 个坐标更省事、更不易漏。

第四步：越界就跳过，最后统计

标记前先判断新坐标是否落在 1 到 n 之间，超出方阵的直接忽略。全部处理完后，遍历整张表，数一数还有多少格仍是 false，就是会生成怪物的格子数。

本题唯一的失分点

火把的照明范围是「十字 + 四角」的不规则形状，不是 5 × 5 正方形。把火把当萤石处理（或相反），样例就会对不上 —— 样例 5 1 0 + (3,3) 的答案 12 正是用来验证这一点的。

例题七 · 参考代码


```
bool light[110][110];           // 是否被照亮
// 火把的 13 个相对偏移：中心十字 + 四个对角
int tx[13] = {0, 0, -1, -1, -1, 0, 0, 0, 1, 1, 1, -2, 2};
int ty[13] = {-2, 2, -1, 0, 1, -2, -1, 1, -1, 0, 1, 0, 0};

int main() {
    int n, m, k;
    scanf("%d %d %d", &n, &m, &k);

    for (int t = 0; t < m; t++) {         // 处理每个火把
        int x, y;
        scanf("%d %d", &x, &y);
        for (int d = 0; d < 13; d++) {
            int nx = x + tx[d], ny = y + ty[d];
            if (nx >= 1 && nx <= n && ny >= 1 && ny <= n)
                light[nx][ny] = true;
        }
    }

    for (int t = 0; t < k; t++) {         // 处理每个萤石
        int x, y;
        scanf("%d %d", &x, &y);
        for (int i = -2; i <= 2; i++)
            for (int j = -2; j <= 2; j++) {
                int nx = x + i, ny = y + j;
                if (nx >= 1 && nx <= n && ny >= 1 && ny <= n)
                    light[nx][ny] = true;
            }
    }
    int ans = 0;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (light[i][j] == false) ans++;
    printf("%d
", ans);
    return 0;
}
```

偏移数组写法

把 13 个相对坐标写成两个平行的常量数组，用一层循环统一处理，比手写 13 个 if 语句整洁得多，也更不容易漏格。

萤石用双重循环

5 × 5 是规则形状，用 i、j 从 −2 到 2 的嵌套循环生成偏移最自然，不必再手写 25 个坐标。

越界判断的位置

每个候选格子都要先判断横纵坐标是否都落在 1 到 n 之间，通过后才写入 light。漏掉这一步会越界写坏内存。

为什么数组开到 110

n 最大 100，而偏移量最小到 −2，理论上访问范围是 −1 到 102。开 110 留足余量；同时越界判断保证了实际只访问 1 到 n。两道保险都做上，最稳妥。

七题总结：一张表带回去

信息学竞赛培训 · 洛谷七题解析

考点 · 写法 · 坑点

把七道题的核心信息压缩成一张表。复习时先看「核心写法」一列，能默写出来就说明这一题真的掌握了。

P5730	显示屏	二维数组 · 打表	按行拼接：外层行、内层数字	数字 1 右对齐；末位不补间隔点
P1428	小鱼比可爱	数组遍历 · 计数	双重循环，内层 j 严格小于 i	比较必须是严格小于；cnt 要每只重置
P1097	统计数字	排序 · 块状计数	排序后一趟扫描，同组累加异组输出	最后一组会漏输出；数值要用 long long
P1616	疯狂的采药	完全背包	内层容量正序枚举（j 从 a[i] 增到 t）	写成倒序就退化成 01 背包；dp 要开全局
P1049	装箱问题	01 背包变式	体积即价值，内层容量倒序枚举	答案要输出 V 减 dp[V]，不是 dp[V]
P5723	质数口袋	贪心 · 筛法	从小到大取质数，边取边累加	必须「先判断再累加」，否则多装一个
P1789	插火把	二维数组 · 模拟	按偏移逐格标记 true，最后数 false	火把范围是不规则的十字加四角

一句话带走

背包看方向（01 倒序、完全正序）；计数看边界（首尾两端最容易漏）；模拟看形状（题目怎么画就怎么标）。

看清题型，比写对代码更重要

七道题，五个专题，一套通用的读题方法

课后任务 · 每题手敲一遍并提交，重点对照「最易错的一点」自查

下一步方向 · 多重背包 · 分组背包 · 区间 DP · 树形 DP

洛谷七题解析 · 完